

CODE EXPLANATION

Code:

```
int pirPin = 2;
int ledPin = 8;
int motionState = 0;

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  motionState = digitalRead(pirPin);
  if (motionState == HIGH) {
    digitalWrite(ledPin, HIGH);
    Serial.println("Motion Detected - LED ON");
  } else {
    digitalWrite(ledPin, LOW);
    Serial.println("No Motion - LED OFF");
  }
  delay(200);
}
```

Code Explanation

Line 1–3 (Variable declarations): Three integer variables are declared. pirPin is assigned to analog pin 2, which is where the PIR sensor's OUT pin is connected. ledPin is assigned to digital pin 8, which controls the LED. motionState stores the current reading from the PIR sensor.

Lines 5–9 (setup() function): This function runs once when the Arduino is powered on.

pinMode(pirPin, INPUT) configures pin 2 to receive signals from the PIR sensor. pinMode(ledPin, OUTPUT) configures pin 8 to send signals to the LED. Serial.begin(9600) starts the Serial Monitor at 9600 baud rate, allowing us to read messages from the Arduino on our computer screen.

Lines 11–21 (loop() function): This function runs repeatedly as long as the Arduino is powered. digitalRead(pirPin) reads the current state of the PIR sensor it returns HIGH (1) if motion is detected, or LOW (0) if no motion is detected. The result is stored in motionState.

Lines 13–19 (if-else statement): If motionState equals HIGH, the Arduino sets pin 8 to HIGH using digitalWrite(), which turns the LED on. It also prints 'Motion Detected - LED ON' to the Serial Monitor. If motionState equals LOW, pin 8 is set to LOW, turning the LED off, and 'No Motion - LED OFF' is printed to the Serial Monitor.

Line 20 (delay): delay(200) pauses the program for 200 milliseconds before the loop repeats. This prevents the Serial Monitor from being flooded with too many messages too quickly, making the output easier to read.